CME Prediction System: Checklist & Troubleshooting

✅ Pre-Implementation Checklist

Environment Setup

- [ ] Python 3.8+ installed
- [ ] Virtual environment created: `python -m venv cme_env`
- [ ] Virtual environment activated: `source cme_env/bin/activate` (Linux/Mac) or `cme_env\Scripts\activate` (Windows)
- [ ] Requirements installed:

```bash
pip install pandas numpy scikit-learn xgboost tensorflow matplotlib seaborn s
```

Data Preparation

- [ ] SWIS Level-1 data downloaded from ISSDC (https://issdc.gov.in/)
- [ ] SoLEX CSV file saved to `data/raw/SoLEX_data.csv`
- [ ] HELIOS CSV file saved to `data/raw/HELIOS_data.csv`
- [ ] CME event labels compiled in `data/raw/cme_events.csv`
  - Format: `timestamp_start, timestamp_end, class`
  - Example:

```
timestamp_start,timestamp_end,class
2023-01-15 10:30:00,2023-01-15 11:45:00,M
2023-01-20 14:20:00,2023-01-20 15:10:00,X
```

Directory Structure

- [ ] Created: `data/raw/`
- [ ] Created: `data/processed/`
- [ ] Created: `data/splits/`
- [ ] Created: `models/`
- [ ] Created: `results/`
- [ ] Created: `notebooks/`

🚀 Step-by-Step Implementation Checklist

Phase 1: Data Loading (Days 1–2)

Goal: Merge SoLEX and HELIOS data into a single CSV

Tasks:

☐ **1.1** Check SWIS CSV columns

```python
import pandas as pd
solex = pd.read_csv('data/raw/SoLEX_data.csv')
print(solex.columns)
print(solex.head())
```

Expected: Columns like `Timestamp, Flux_0.5-1.0keV, Flux_1.0-2.0keV, ...`

☐ **1.2** Check for timestamp format mismatches

```python
print(solex['Timestamp'].dtype)
solex['Timestamp'] = pd.to_datetime(solex['Timestamp'])  # Convert if needed
```

☐ **1.3** Merge on Timestamp

```python
merged = pd.merge(solex, helios, on='Timestamp', how='inner', suffixes=('_sol
print(f"Merged shape: {merged.shape}")
```

Expected: `(n_rows, 20+)` — should have both soft and hard X-ray channels

☐ **1.4** Save merged file

```python
merged.to_csv('data/raw/merged_swis.csv', index=False)
```

Phase 2: Preprocessing (Days 3–5)

Goal: Clean data, engineer features, create labels

Tasks:

☐ **2.1** Load merged data and check quality

```python
df = pd.read_csv('data/raw/merged_swis.csv', parse_dates=['Timestamp'])
print(f"Shape: {df.shape}")
print(f"Missing values:\n{df.isnull().sum()}")
```

Expected: Some NaN is OK; >50% missing → need better data source

☐ **2.2** Identify flux columns

```python
flux_cols = [c for c in df.columns if 'Flux' in c or 'flux' in c]
print(f"Found {len(flux_cols)} flux columns")
```

Expected: 8–16 columns

☐ **2.3** Handle missing values

```python
df[flux_cols] = df[flux_cols].fillna(method='ffill').fillna(method='bfill')
df = df.dropna(subset=flux_cols)
```

Expected: Fewer rows, but no NaN in flux columns

☐ **2.4** Normalize flux data

```python
from sklearn.preprocessing import RobustScaler
scaler = RobustScaler()
df[flux_cols] = scaler.fit_transform(df[flux_cols])
print(f"Normalized range: [{df[flux_cols].min().min():.3f}, {df[flux_cols].ma
```

Expected: Values roughly in [-2, 2] range (robust scaling)

☐ **2.5** Create features

```python
```

```
# Derivatives, rolling stats, ratios (see code in Phase 2)
df['SoLEX_Helios_ratio'] = df[solex_cols].mean(axis=1) / df[helios_cols].mean
```

Expected: Total features: 50–100

☐ **2.6** Create CME labels

python

```
# Check CME event file
cme_events = pd.read_csv('data/raw/cme_events.csv', parse_dates=['timestamp_s
print(f"Loaded {len(cme_events)} CME events")
print(cme_events.head())
```

Troubleshooting: If no CME events file:

Download from GOES catalog:
https://www.sws.bom.gov.au/Solar/IPS/coronal_mass_ejections
Or use NOAA SWPC: https://www.swpc.noaa.gov/

☐ **2.7** Check class balance

python

```
print(f"CME samples: {y.sum()}")
print(f"Non-CME: {len(y) - y.sum()}")
print(f"Ratio: {(len(y) - y.sum()) / y.sum():.1f}:1")
```

Expected: 10:1 to 100:1 imbalance (this is normal for rare events)

☐ **2.8** Create train/val/test splits

python

```
# Chronological split (important!)
df_train = df[:int(0.6*len(df))]
df_val = df[int(0.6*len(df)):int(0.8*len(df))]
df_test = df[int(0.8*len(df)):]
```

Expected: Each split has >100 samples; test has >5 CME events

---

Phase 3: Random Forest Baseline (Days 5–6)

Goal: Establish baseline performance

Tasks:

☐ **3.1** Extract features and labels

```python
feature_cols = [c for c in df_train.columns if c not in ['Timestamp', 'label'
X_train = df_train[feature_cols].values
y_train = df_train['label'].values
print(f"Features: {X_train.shape}")
print(f"Labels: {y_train.shape}")
```

Expected: X shape `(n_samples, 50-100)`, y shape `(n_samples,)`

☐ **3.2** Train Random Forest

```python
from sklearn.ensemble import RandomForestClassifier
rf = RandomForestClassifier(n_estimators=100, max_depth=15,
                            class_weight='balanced', n_jobs=-1)
rf.fit(X_train, y_train)
```

Expected: Should complete in <5 minutes on modern hardware

☐ **3.3** Evaluate on all splits

```python
from sklearn.metrics import roc_auc_score, precision_score, recall_score

for name, X, y in [('Train', X_train, y_train), ('Val', X_val, y_val), ('Test
    y_pred_proba = rf.predict_proba(X)[:, 1]
    print(f"{name}: ROC-AUC={roc_auc_score(y, y_pred_proba):.4f}")
```

Expected: Train > Val ≈ Test; ROC-AUC > 0.80

☐ **3.4** Check for overfitting

```python
```

```
train_auc = roc_auc_score(y_train, rf.predict_proba(X_train)[:, 1])
val_auc = roc_auc_score(y_val, rf.predict_proba(X_val)[:, 1])
print(f"Overfitting gap: {train_auc - val_auc:.4f}")
```

Expected: Gap < 0.10 is good; gap > 0.15 → increase max_depth or reduce n_estimators

☐ **3.5** Save model

python

```
import joblib
joblib.dump(rf, 'models/random_forest.pkl')
```

Phase 4: XGBoost (Days 7–8)

Goal: Improve precision over Random Forest

Tasks:

☐ **4.1** Calculate scale_pos_weight

python

```
n_neg = (y_train == 0).sum()
n_pos = (y_train == 1).sum()
scale_pos_weight = n_neg / n_pos
print(f"scale_pos_weight = {scale_pos_weight:.2f}")
```

Expected: Typically 10–50 for rare events

☐ **4.2** Train XGBoost

python

```
import xgboost as xgb
xgb_model = xgb.XGBClassifier(scale_pos_weight=scale_pos_weight,
                              max_depth=6, learning_rate=0.05)
xgb_model.fit(X_train, y_train,
              eval_set=[(X_val, y_val)],
              early_stopping_rounds=20,
              verbose=10)
```

Expected: Finishes in 2–10 minutes; best_iteration < n_estimators

☐ **4.3** Compare to Random Forest

```python
rf_auc = roc_auc_score(y_test, rf.predict_proba(X_test)[:, 1])
xgb_auc = roc_auc_score(y_test, xgb_model.predict_proba(X_test)[:, 1])
print(f"RF:  {rf_auc:.4f}")
print(f"XGB: {xgb_auc:.4f}")
print(f"Improvement: {xgb_auc - rf_auc:+.4f}")
```

Expected: XGBoost improves by 0.01–0.03

☐ **4.4** Check precision improvement

```python
from sklearn.metrics import precision_score

rf_prec = precision_score(y_test, rf.predict(X_test))
xgb_prec = precision_score(y_test, xgb_model.predict(X_test))
print(f"RF Precision:  {rf_prec:.4f}")
print(f"XGB Precision: {xgb_prec:.4f}")
```

Expected: XGB precision >0.70 (your main weakness fix)

☐ **4.5** Save model

```python
xgb_model.save_model('models/xgboost.json')
```

Phase 5: LSTM (Days 9–12)

Goal: Capture temporal patterns in solar wind data

Tasks:

☐ **5.1** Create sliding windows

```python
```

```
def create_windows(X, y, window_size=30, step_size=5):
    X_w, y_w = [], []
    for i in range(0, len(X) - window_size, step_size):
        X_w.append(X[i:i+window_size])
        y_w.append(y[i:i+window_size].max())
    return np.array(X_w), np.array(y_w)

X_train_3d, y_train_3d = create_windows(X_train, y_train)
print(f"3D shape: {X_train_3d.shape}")  # Should be (n_windows, 30, n_feature
```

Expected: X_train_3d.shape = `(~300, 30, ~50)` for typical dataset

☐ **5.2** Check window quality

python

```
print(f"Train windows: {X_train_3d.shape[0]} (CME: {y_train_3d.sum()})")
print(f"Val windows:   {X_val_3d.shape[0]} (CME: {y_val_3d.sum()})")
print(f"Test windows:  {X_test_3d.shape[0]} (CME: {y_test_3d.sum()})")
```

Expected: At least 200 training windows; >10 CME windows

☐ **5.3** Build LSTM model

python

```python
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dropout, Dense, Bidirectional

model = Sequential([
    Bidirectional(LSTM(64, return_sequences=True,
                       input_shape=(X_train_3d.shape[1], X_train_3d.shape[2]))),
    Dropout(0.2),
    Bidirectional(LSTM(32, return_sequences=False)),
    Dropout(0.2),
    Dense(16, activation='relu'),
    Dropout(0.2),
    Dense(1, activation='sigmoid')
])

model.compile(optimizer='adam', loss='binary_crossentropy',
              metrics=['AUC', 'Precision', 'Recall'])
print(model.summary())
```

Expected: ~250K parameters; summary should show layer shapes

☐ **5.4** Train LSTM

```python
from tensorflow.keras.callbacks import EarlyStopping

early_stop = EarlyStopping(monitor='val_loss', patience=10, restore_best_weig
history = model.fit(X_train_3d, y_train_3d,
                    validation_data=(X_val_3d, y_val_3d),
                    epochs=50, batch_size=32, callbacks=[early_stop], verbose
```

Expected: Training completes in <10 minutes on GPU; val_loss decreases for first 20-30 epochs

☐ **5.5** Plot training history

```python
```

```
import matplotlib.pyplot as plt
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Val Loss')
plt.legend()
plt.savefig('results/lstm_history.png')
```

Expected: Loss curves should decrease, val_loss shouldn't increase sharply

☐ **5.6** Evaluate LSTM

python

```
loss, auc, prec, rec = model.evaluate(X_test_3d, y_test_3d)
y_lstm_proba = model.predict(X_test_3d).flatten()
lstm_auc = roc_auc_score(y_test_3d, y_lstm_proba)
print(f"LSTM ROC-AUC: {lstm_auc:.4f}")
```

Expected: ROC-AUC > 0.85

☐ **5.7** Save LSTM model

python

```
model.save('models/lstm.h5')
```

Phase 6: Anomaly Detection (Days 12–13)

Goal: Create an extra feature for the ensemble

Tasks:

☐ **6.1** Train Isolation Forest

python

```
from sklearn.ensemble import IsolationForest

iso_forest = IsolationForest(contamination=0.05, random_state=42)
iso_forest.fit(X_train)
```

Expected: Completes in <1 minute

☐ **6.2** Score all samples

```python
anom_scores = iso_forest.score_samples(X_test)
print(f"Anomaly scores range: [{anom_scores.min():.3f}, {anom_scores.max():.3
```

Expected: More negative = more anomalous

- [ ] **6.3** Normalize scores to [0, 1]

```python
anom_norm = (anom_scores - anom_scores.min()) / (anom_scores.max() - anom_sco
print(f"Normalized range: [{anom_norm.min():.3f}, {anom_norm.max():.3f}]")
```

- [ ] **6.4** Save model

```python
joblib.dump(iso_forest, 'models/isolation_forest.pkl')
```

Phase 7: Stacking Ensemble (Days 14–15)

Goal: Combine all 3 base models into 1 meta-model

Tasks:

- [ ] **7.1** Get predictions from all base models

```python
y_rf_val = rf.predict_proba(X_val)[:, 1]
y_xgb_val = xgb_model.predict_proba(X_val)[:, 1]

# LSTM predictions need 3D input (from last 30 timesteps)
X_val_3d, _ = create_windows(X_val, y_val, window_size=30, step_size=1)
y_lstm_val = model.predict(X_val_3d, verbose=0).flatten()

print(f"RF:   {y_rf_val.shape}")
print(f"XGB:  {y_xgb_val.shape}")
print(f"LSTM: {y_lstm_val.shape}")
```

Expected: All same shape `(n,)`

☐ **7.2** Handle shape mismatch (LSTM produces fewer samples)

```python
# If LSTM has fewer windows, align to RF/XGB length
# Solution: use only the matching indices

# Or: pad LSTM predictions to match
if len(y_lstm_val) < len(y_rf_val):
    y_lstm_val = np.concatenate([y_lstm_val, [0.5] * (len(y_rf_val) - len(y_l
```

Expected: All predictions same length

☐ **7.3** Stack into meta-features

```python
X_meta_val = np.column_stack([y_rf_val, y_xgb_val, y_lstm_val])
X_meta_test = np.column_stack([y_rf_test, y_xgb_test, y_lstm_test])

print(f"Meta-features shape: {X_meta_val.shape}")  # Should be (n, 3)
```

☐ **7.4** Train meta-model

```python
from sklearn.linear_model import LogisticRegression

meta_model = LogisticRegression(class_weight='balanced', max_iter=1000)
meta_model.fit(X_meta_val, y_val)

weights = meta_model.coef_[0]
print(f"Weights: RF={weights[0]:.3f}, XGB={weights[1]:.3f}, LSTM={weights[2]:
```

Expected: Weights sum to ~1.0 (due to regularization); should favor high-performing models

☐ **7.5** Evaluate ensemble on test

```python
```

```python
y_ensemble_proba = meta_model.predict_proba(X_meta_test)[:, 1]
ensemble_auc = roc_auc_score(y_test, y_ensemble_proba)
print(f"Ensemble ROC-AUC: {ensemble_auc:.4f}")
```

Expected: Should be > individual model AUCs

☐ **7.6** Save meta-model

```python
joblib.dump(meta_model, 'models/meta_model.pkl')
```

Phase 8: Evaluation (Day 16)

Goal: Compare all models and generate final report

Tasks:

☐ **8.1** Create comparison table

```python
```

```python
import pandas as pd

results = {}
for name, y_pred_proba in [
    ('RF', y_rf_test),
    ('XGB', y_xgb_test),
    ('LSTM', y_lstm_test),
    ('Ensemble', y_ensemble_proba)
]:
    y_pred = (y_pred_proba > 0.5).astype(int)

    results[name] = {
        'ROC-AUC': roc_auc_score(y_test, y_pred_proba),
        'Precision': precision_score(y_test, y_pred, zero_division=0),
        'Recall': recall_score(y_test, y_pred, zero_division=0),
        'F1': 2*(precision*recall)/(precision+recall + 1e-8)
    }

df = pd.DataFrame(results).T
print(df)
df.to_csv('results/final_metrics.csv')
```

8.2 Plot ROC curves

```python
from sklearn.metrics import roc_curve, auc
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 8))
for name, y_pred_proba in [('RF', y_rf_test), ('XGB', y_xgb_test), ('LSTM', y
    fpr, tpr, _ = roc_curve(y_test, y_pred_proba)
    roc_auc = auc(fpr, tpr)
    plt.plot(fpr, tpr, label=f'{name} (AUC={roc_auc:.3f})', linewidth=2)

plt.plot([0, 1], [0, 1], 'k--', label='Random')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curves - All Models')
plt.legend()
plt.grid(alpha=0.3)
plt.savefig('results/roc_curves.png', dpi=300, bbox_inches='tight')
```

☐ **8.3** Plot confusion matrices

```python
from sklearn.metrics import confusion_matrix
import seaborn as sns

fig, axes = plt.subplots(1, 4, figsize=(16, 3))

for idx, (name, y_pred_proba) in enumerate([('RF', y_rf_test), ('XGB', y_xgb_
    y_pred = (y_pred_proba > 0.5).astype(int)
    cm = confusion_matrix(y_test, y_pred)

    sns.heatmap(cm, annot=True, fmt='d', ax=axes[idx], cmap='Blues')
    axes[idx].set_title(name)
    axes[idx].set_ylabel('True')
    axes[idx].set_xlabel('Predicted')

plt.tight_layout()
plt.savefig('results/confusion_matrices.png', dpi=300, bbox_inches='tight')
```

🔧 Troubleshooting Guide

Problem: "No module named 'xgboost'"

Solution:

```bash
pip install xgboost --upgrade
```

Problem: "No module named 'tensorflow'"

Solution:

```bash
pip install tensorflow --upgrade

# Or, if you have GPU support (NVIDIA):
pip install tensorflow[and-cuda]
```

Problem: "CSV file not found"

Solution:

1. Check file paths:

```python
import os
print(os.listdir('data/raw/'))  # List files in directory
```

2. Make sure paths are absolute or relative from the correct directory:

```python
import os
os.chdir('/path/to/project')  # Change working directory
```

Problem: "No columns match 'Flux'"

Solution:

1. Check actual column names:

```python
df = pd.read_csv('data/raw/SoLEX_data.csv')
print(df.columns)
```

2. Adjust feature column detection:

```python
flux_cols = [c for c in df.columns if 'intensity' in c.lower() or 'count' in
```

Problem: "KeyError: 'Timestamp'"

Solution:

1. Check if Timestamp column exists:

```python
```

```
print(df.columns)
```

2. If column has different name:

```python
df = df.rename(columns={'time': 'Timestamp'})
```

3. If timestamp in index:

```python
df = df.reset_index()
df = df.rename(columns={'index': 'Timestamp'})
```

Problem: "Shapes mismatch when stacking predictions"

Solution:

```python
# LSTM produces (n_windows,) while RF/XGB produce (n_samples,)
# Align them:

min_len = min(len(y_rf_test), len(y_xgb_test), len(y_lstm_test))
y_rf_test = y_rf_test[:min_len]
y_xgb_test = y_xgb_test[:min_len]
y_lstm_test = y_lstm_test[:min_len]

X_meta_test = np.column_stack([y_rf_test, y_xgb_test, y_lstm_test])
```

Problem: "ROC-AUC is too low (<0.75)"

Diagnosis & Solutions:

1. **Check data quality:**

```python
print(f"Missing values: {df.isnull().sum().sum()}")
print(f"Duplicate timestamps: {df.duplicated(subset=['Timestamp']).sum()}")
```

2. **Check class imbalance:**

```python
print(f"Class balance: {y.sum()} / {len(y)}")  # Should be <10% positive
```

3. **Try better hyperparameters:**

```python
# For Random Forest
rf = RandomForestClassifier(n_estimators=200, max_depth=20, min_samples_spli

# For XGBoost
xgb_model = xgb.XGBClassifier(scale_pos_weight=50, max_depth=8, learning_rat
```

4. **Engineer better features:**
   - Add more rolling windows (3, 7, 15 minute windows)
   - Add spectral features (FFT components)
   - Add gradient magnitude: `np.sqrt(diff1² + diff2²)`

Problem: "LSTM training too slow"

Solution:

```bash
# Install GPU support
pip install tensorflow[and-cuda]

# Or use smaller LSTM:
model = Sequential([
    LSTM(32, return_sequences=True, input_shape=(window_size, n_features)),
    Dropout(0.2),
    LSTM(16),
    Dropout(0.2),
    Dense(1, activation='sigmoid')
])
```

Problem: "Overfitting: Train AUC >> Val AUC"

Solution:

```python
```

```
# Reduce model complexity
rf = RandomForestClassifier(max_depth=10, min_samples_split=10)

# Or add more regularization
xgb_model = xgb.XGBClassifier(
    reg_lambda=2.0,  # Increase
    reg_alpha=1.0,   # Increase
    subsample=0.7,   # Decrease
    colsample_bytree=0.7  # Decrease
)
```

Problem: "Precision is too low (<0.60)"

Solution:

1. **Adjust decision threshold:**

```
python
    y_pred = (y_pred_proba > 0.7).astype(int)  # Higher threshold = higher preci
```

2. **Weight positive class higher:**

```
python
    xgb_model = xgb.XGBClassifier(scale_pos_weight=100)  # Increase weight
```

3. **Use Precision-Recall trade-off curve:**

```
python
    from sklearn.metrics import precision_recall_curve

    precision, recall, thresholds = precision_recall_curve(y_test, y_pred_proba)

    # Find threshold that maximizes F1
    f1_scores = 2 * (precision * recall) / (precision + recall + 1e-8)
    best_threshold = thresholds[np.argmax(f1_scores)]
    y_pred_tuned = (y_pred_proba > best_threshold).asint(int)
```

Problem: "CME event file has wrong timestamp format"

Solution:

```python
# Try different formats
cme_events['timestamp_start'] = pd.to_datetime(cme_events['timestamp_start'], f

# Or
cme_events['timestamp_start'] = pd.to_datetime(cme_events['timestamp_start'], f

# Check result
print(cme_events['timestamp_start'].dtype)  # Should be datetime64
```

📊 Expected Final Results

Realistic performance targets (after optimization):

Metric	Target	Achievable with Ensemble
ROC-AUC	> 0.85	✅ 0.86–0.89
Precision	> 0.70	✅ 0.72–0.78
Recall	> 0.70	✅ 0.71–0.79
F1-Score	> 0.70	✅ 0.71–0.78
False Positive Rate	< 0.10	✅ 0.07–0.09

Model ranking (typical):

1. **Ensemble (Best)** — combines strengths of all 3
2. **XGBoost** — excellent precision
3. **LSTM** — excellent recall
4. **Random Forest** — balanced baseline

🎯 Quick Wins (If Running Behind Schedule)

If you have limited time, prioritize:

1. **Skip Isolation Forest** → Use RF + XGB only
2. **Skip LSTM early stopping** → Just train for 20 epochs
3. **Use simple ensemble** → Average probabilities instead of LogReg:

```python
y_ensemble = (y_rf + y_xgb + y_lstm) / 3
```

4. **Minimal feature engineering** → Just use normalized flux + 1 rolling window

Minimum viable pipeline (1 week):

1. Data loading + preprocessing → 2 days
2. Train RF → 1 day
3. Train XGB → 1 day
4. Simple ensemble → 1 day
5. Evaluation + dashboard → 1 day

✨ Final Submission Checklist

- [ ] All 4 models trained and saved
- [ ] Test metrics calculated and documented
- [ ] ROC curves plotted
- [ ] Confusion matrices generated
- [ ] Code commented and documented
- [ ] Results saved to CSV
- [ ] README.md created explaining how to run
- [ ] Inference pipeline tested on new data
- [ ] Nowcasting catalog generated (sample)
- [ ] Dashboard prototype created

Good luck! 🤗

For questions or issues, refer back to the **CME_Implementation_Plan.md** for detailed explanations.